The Pacific Wire

VOL. 1, NO. 073

2026-03-30

FRONT PAGE

Nutrition Advice Decoded: What Foods Are Actually Good For Us, What Should We Avoid, and Why Is It All SO Confusing?

Cynthia Graber and Nicola Twilley · Gastropod

Are eggs going to give you high cholesterol, or are they the base of a great protein-rich meal? Will coffee give you cancer, or will it help you live longer? If you're confused about what nutrition science has to say about which foods are healthy and which are not, you're not alone. But why is it so hard to figure out what's good for us, and why does the advice seem to change constantly? This week on Gastropod, we reveal the hidden history of how nutrition science got started, why its early successes saved millions of lives, and how those successes then led the field astray. From debunking the Blue Zones, to the discredited "French paradox" that had everyone washing their Brie down with glasses of red wine, to the most recent research on ultra-processed foods, we're breaking down how nutrition research actually gets done—and what that means for what should be on your plate.

FEATURES

My Answers for Microservices Awkward Questions

Jay · Jay Fields

Earlier this year, Ade published Awkward questions for those boarding the microservices bandwagon . I think the list is pretty solid, and (with a small push from Ade) I decided to write concise details on my experience.

I think it's reasonable to start with a little context building. When I started working on the application I'm primarily responsible for microservices were very much fringe. Fred George was already giving (great) presentations on the topic, but the idea had gained neither momentum nor hype. I never set out to write "microsevices"; I set out to write a few small projects (codebases) that collaborated to provide a (single) solid user experience.

I pushed to move the team to small codebases after becoming frustrated with a monolith I was a part of building and a monolith I ended up inheriting. I have no idea if several small codebases are the right choice for the majority, but I find they help me write more maintainable software. Practically everything is a tradeoff in software development; when people ask me what the best aspect of a small services approach is, I always respond: I find accidental coupling to be the largest productivity drain on projects with monolithic codebases. Independent codebases reduce what can be reasonably accidentally coupled.

As I said, I never set out to create microservices; I set out to reduce accidental coupling. 3 years later, it seems I have around 3 years of experience working with Microservies (according to the wikipedia definition). I have nothing to do with microservices advocacy, and you shouldn't see this entry as confirmation or condemnation of a microservices approach. The entry is an experience report, nothing more, nothing less.

On to Ade's questions (in bold).

Why isn't this a library? :

It is. Each of our services compile to a jar that can be run independently, but could just as easily be used as a library.

What heuristics do you use to decide when to build (or extract) a service versus building (or extracting) a library? :

Something that's strictly a library provides no business value on it's own.

How do you plan to deploy your microservices? :

Shell scripts copy jars around, and we have a web UI that provides easy deployment via a browser on your laptop, phone, or anywhere else.

What is your deployable unit? :

A jar.

Will you be deploying each microservice in isolation or deploying the set of microservices needed to implement some business functionality? :

In isolation. We have a strict rule that no feature should require deploying separate services at the same time. If a new feature requires deploying new versions of two services, one of them must support old and new behavior - that one can be deployed first. After it's run in production for a day the other service can be deployed. Once the second service has run in production for a day the backwards compatibility for the first service can be safely removed.

Are you capable of deploying different instances (where an instance may represent multiple processes on multiple machines) of the same microservice with different configurations? :

If I'm understanding your question correctly, absolutely. We currently run a different instance for each trading desk.

Is it acceptable for another team to take your code and spin up another instance of your microservice? :

Yes.

Can team A use team B's microservice or are they only used within rather than between teams? :

Anyone can request a supported instance or fork individual services. If a supported instance is requested the requestor has essentially become a customer, a peer of the trading desks, and will be charged the appropriate costs allocated to each customer. Forking a service is free, but comes with zero guarantees. You fork it, you own it.

Do you have consumer contacts for your microservices or is it the consumer's responsibility to keep up with the changes to your API? :

It is the consumer's responsibility.

Is each microservice a snowflake or are there common conventions? :

There are common conventions, and yet, each is also a snowflake. There are common libraries that we use, and patterns that we follow, but many services require sightly different behavior. A service owner (or primary) is free to make whatever decision best fits the problem space.

How are these conventions enforced? :

Each service has a primary, who is responsible for ensuring consistency (more here). Before you become a primary you'll have spent a significant amount of time on the team; thus you'll be equipped with the knowledge required to apply conventions appropriately.

How are these conventions documented? : They are documented strictly by the code.

Philopappou Hill Pathways in Athens, Greece

Atlas Obscura

Philopappou Hill takes its name from Prince Gaius Julius Antiochus Epiphanes Philopappos. Philopappos came to Greece in the 1 st century A. D. E. from the Kingdom of Commagene after it was assimilated into the Roman Empire. Philopappos was extremely wealthy and was known for his philanthropy, funding many projects in the city. When Philopappos died in 116 AD his sister built his tomb as a monument directly southwest of the Acropolis atop what was then known as the Hill of the Muses. The Philopappos' monument was a magnificent, two-story structure, built with solid Pentelic and Hymettian marble. The monument remained intact until at 15th century, when portions were stripped by the occupying Ottomans for various uses. The remains of the monument sat on Philopappos Hill in that condition for the next 500+ years.

In 1951 the Greek government developed a plan to beautify and make the area south of the Acropolis more accessible to the citizens of Athens. The project included pedestrian paths that led to the summit of Philopappos Hill. Local architect Dimitris Pikionis was selected to oversee the project. Pikionis was born in Piraeus, Greece in 1887. He was interested in building design as a youth and eventually enrolled at the National Technical University of Athens to study civil engineering. He graduated in 1908 and left Greece to continue his studies in France and Germany. He returned to Greece in 1912 and began focusing his studies on modern Greek architecture. In 1921 he accepted a position as a lecturer at the National Technical University of Athens and in 1925 accepted a position as a full professor. He worked on various projects throughout the city while teaching, and in 1951 he was selected by the Greek government to lead the efforts to implement the project.

The project began in 1954. Pikionis collaborated with his colleagues, students and local stonemasons and together they developed a plan that was mostly reliant upon impromptu on-site decisions. The approach did not involve detailed written plans, nor was there a need for approval from the Greek government.

At the time Philopappou Hill was a mostly barren, rocky hill with narrow dirt paths leading to the summit. Pikionis' vision for the hill was to reforest it with a large number of pine and olive trees, insert steps, pave the paths, and install numerous seating areas with views of the nearby Acropolis.

A collage of ancient ruins, cut marble, salvaged stone pieces from demolished neoclassical buildings, and ceramic shards found in the area were then gathered and organized. The students and workers then shaped the objects with chisels and carefully cemented them in a collage into the numerous paths leading to rest areas and the summit. The merging of all these objects created what many considered an amazing piece of art.

The project was completed with little fanfare in February 1958 and included the pedestrian walkway leading to the Propylaea and the area surrounding the Church of Agios Dimitrios Loumbardiaris. Despite the low-key approach, this project was recognized as part of the Acropolis of Athens when it was added to the UNESCO World Heritage List in 1987. This project is considered by many to be one of the most significant landscape architecture works in the 20th-century.

Using Kamal 2.0 in Production

Fly.io Blog

Agile Web Development with Rails 8 is off to production, where they do things like editing, indexing, pagination, and printing. In researching the chapter on Deployment and Production, I became very dissatisfied with the content available on Kamal. I ended up writing my own, and it went well beyond the scope of the book. I then extracted what I needed from the result and put it in the book.

Now that I have some spare time, I took a look at the greater work. It was more than a chapter and less than a book, so I decided to publish it online .

This took me only a matter of hours. I had my notes in the XML grammar that Pragmatic Programming uses for books. I asked GitHub Copilot to convert them to Markdown. It did the job without my having to explain the grammar. It made intelligent guesses as to how to handle footnotes and got a number of these wrong, but that was easy to fix. On a lark, I asked it to proofread the content, and it did that too.

Don't get me wrong, Kamal is great. There are plenty of videos on how to get toy projects online, and the documentation will tell you what each field in the configuration file does. But none pull together everything you need to deploy a real project. For example, there are seven things you need to get started . Some are optional, some you may already have, and all can be gathered quickly if you have a list .

Kamal is just one piece of the puzzle. To deploy your software using Kamal, you need to be aware of the vast Docker ecosystem. You will want to set up a builder, sign up for a container repository, and lock down your secrets. And as you grow, you will want a load balancer and a managed database.

And production is much more than copying files and starting a process. It is ensuring that your database is backed up, that your logs are searchable, and that your application is being monitored. It is also about ensuring that your application is secure.

My list is opinionated. Each choice has a lot of options. For SSH keys, there are a lot of key types. If you don't have an opinion, go with what GitHub recommends. For hosting, there are a lot of providers, and most videos start with Hetzner, which is a solid choice. But did you know that there are separate paths for Cloud and Robot, and when you would want either?

A list with default options and alternatives highlighted was what would have helped me get started. Now it is available to you. The source is on GitHub . CC0 licensed . Feel free to add side pages or links to document Digital Ocean, add other monitoring tools, or simply correct a typo that GitHub Copilot and I missed (or made).

And if you happen to be in the south eastern part of the US in August, come see me talk on this topic at the Carolina Code Conference . If you can't make it, the presentation will be recorded and posted online.

Making Remote Work: Tools

Jay · Jay Fields

I recently wrote about my experiences working on a remote team . Within that blog entry you can find a more verbose version of the following text:

Communication is what I consider to be the hardest part of remote work. I haven't found an easy, general solution. A few teammates prefer video chat, others despise it. A few teammates like the wiki as a backlog, a few haven't ever edited the wiki. Some prefer strict usage of email/chat/phone for async-unimportant/async-important/sync-urgent, others tend to use one of those 3 for all communication.

As you can tell, we have several different communication tools. When writing, I generally prefer to include concrete examples. This blog entry will list each tool referenced above. However, I cannot emphasize enough that: this list is a snapshot of what we're using, not a recommended set of tools .

app: Github

usage: We use many of the features of Github; however, the two features that help facilitate remote work are (a) pull requests with inline comments and (b) compare . A pull request with inline comments has (thus far) been the most productive way to asynchronously discuss specific pieces of code. Almost all non-trivial commits will eventually end up in a pull request that's reviewed by at least one other team member. We've found compare view to be the best solution for distilling changes for a teammate with limited context.

app: Hipchat

usage: We have 3 hipchat rooms: work, social, support. It should be pretty obvious what we use each room for. The primary driver for splitting the 3 is for keeping noise down. Most team members look at chat history for work and support, reading anything that happened between now and the last time they were logged in. Social tends to be more verbose, often off-topic, and never required reading for keeping up with what the team is up to.

app: Cisco Jabber

usage: Within the team, we primarily use Cisco Jabber for video calls; however Cisco Jabber is also a great way for people within DRW offices to reach anyone on my team without having to know their location. Cisco Jabber is significantly better than asking people to remember to call your cell, or forwarding your desk phone to your cell - it provides you 1 number that anyone can reach you at, regardless of your physical location. There's not much to say about the video capabilities, they're there, they work well. Cisco Jabber also provides desktop sharing, which we use occasionally for "remote pair-programming".

app: Confluence

usage: Our backlog resides on a Confluence wiki; it's a single page with about 150 lines. The backlog is split into 3 sections: Milestones, Now, and Soon. There are generally 3-5 Milestones, which list (as sub-bullets) their dependencies. A dependency is a reference to a line item that will live in Now or Soon. Now is the list of highest priority tasks - the things we need to get down right away. Soon is the list of things that are urgent but not important, or important but not urgent. Both Now and Soon lists contain placeholders for conversations, each placeholder is around 1-2 lines. Below you'll find a contrived, sample backlog.

Deploy to Billy Ray Valentine

(market data 1)

(execution 1)

Automated Order Matching (stakeholder: Mortimer and Randolph Duke)

(execution 2)

(reporting 1)

Market data

(1) support pork bellies

support orange juice

(1) support pork bellies

(2) internally match incoming orders from customers

Market data

support coffee

support wheat

(1) Commission summary

note: some things in Now need to be done immediately, but do not support an upcoming milestone. These things are incremental changes for previously met milestones.

Obviously we use email and other tools as well, but I can't think of any remote specific usage patterns that are worth sharing.

As I previously mentioned, each member of the team uses each of these tools in their own way. None of these tools are ideal for every member of the team, and I believe a good team lead helps ensure each team member is only required to use the tools they find most helpful.

Experience Report: Weak Code Ownership

Jay · Jay Fields

In 2006 Martin Fowler wrote about Code Ownership . It's a quick read, I'd recommend checking it out if you've never seen it. At the time I was working at ThoughtWorks; I remember thinking "Clearly Strong makes no sense and I have no idea what scenario would make Weak reasonable". 8 years later, I find myself advocating for Weak Code Ownership within my team.

Collective Code Ownership (CCO) served me well between 2005 and 2009. Given the make-up of the teams that I was a part of I found it to be the most effective way to deliver software. Around 2009 I joined a team that eventually grew to around 9 people, all very senior developers. The team practiced Collective Code Ownership. Everyone on the team was very talented, but that didn't translate to constant agreement. In fact, we disagreed far more often than I thought we should. That experience drove me to write about the importance of Compatible Opinions . I still believe in the importance of compatible opinions, but I now wonder if the team wouldn't have been more effective (despite incompatible opinions) if we had adopted Weak Code Ownership.

The 2009 project heavily shaped my approach to developing software. I suspect I'm not the only one who (at one time) believed: if we get a team full of massively talented people we can do anything. It turns out, it's not nearly that easy. Too many cooks in the kitchen is the obvious concern, and it does come up. However, the much larger problem is that talented people work in vastly different ways. Some meticulously refactor in small steps, others make wide reaching and large changes. Some prefer one language to rule them all, others are comfortable switching between 12-15 different languages in the same day. Monolithic vs separated codebases. Inherited vs duplicated config. It goes on and on. You try to optimize for everyone, to ensure everyone is maximally effective. Pretty quickly you run into this situation-

If you optimize everything, you will always be unhappy. – Donald Knuth

Looking back, I believe our "Collective Code Ownership" degraded to "Last in Wins". People began to cluster around shared opinions as the team grew. Inevitably the components of the project splintered and work included constant angling to ensure you only worked in areas designed to match your personal style. Perhaps that wasn't such a bad thing, but never formalizing the splinters led to constant discussion around responsibility and design. Those discussions always felt like waste to me.

I eventually left that team, and I left with the feeling that as a team we'd been ignoring Diffusion of responsibility (DOR) , Bystander effect (BE) , and Crowd psychology (CP) . In my opinion the combination of (exclusively) talented, senior developers and collective code ownership led to suboptimal productivity. The belief that "CCO works and we're just doing it wrong" made things worse. It seemed like our solution was always: we just need to do better. You could write this team off as not as talented as I describe, but you'd be mistaken. All of the developers on the team had great success before the project, and (after that team split up) 8 out of 9 have gone on to lead very successful teams.

Several years later I found myself leading a team that was beginning to grow. We had just expanded to a 4 person team, and I could see the high level problems beginning to appear. Consistency had begun slip, and bugs had begun to appear in places where code was "correct" at the micro level, but the system didn't work as expected at the macro level. There are a few ways to manage these issues, pair programming was an obvious solution to this problem, but I believe pair programming would have introduced a different class of problems. I believe switching to pair programming (exclusively) would actually have been net negative for the team. I've written about giving up on exclusive pair programming , I'll leave it at that. The issues we were facing felt vaguely familiar, and I started to wonder if they stemmed from DOR, BE, and CP. Collective code ownership allowed developers to pop in and out of codebases, making small changes to enable features, but where was the person looking at the big picture? "It's everyone's responsibility" wasn't an answer I was comfortable with.

I found myself looking for another solution that would

encourage consistency

give equal importance to macro and micro quality

address the other issues caused by DOR, BE, and CP

We already had our project split in a microservices style, and I proposed to the team that we move to what I called Primary Code Ownership. Primary Code Ownership can be described as-

All of the developers on the team had great success before the project, and (after that team split up) 8 out of 9 have gone on to lead very successful teams.

Jay

A codebase has a "primary", the person who's responsible for that process in production. You could use the word "owner", but I don't think it conveys what I'm looking for. The team still owns the code, but when problems occur the responsibility falls on the primary first.

The primary drives the architecture of the codebase. The primary gets final say on architectural decisions within a codebase.

A primary may commit to master a codebase without code review.

Any commit to master from a non-primary must come via

Talking Taco

Cynthia Graber and Nicola Twilley · Gastropod

Whether or not it's Tuesday, tacos pretty much always hit the spot—and not just in their homeland in Mexico. These days, the taco can be found all over the world, although it's acquired some strange new fillings along the way, from French fries to canned corn. It's hard to imagine something so universal having to be invented—but, in fact, the taco as we know it only emerged in the 1800s. In this episode, Gastropod talks taco with the experts, peering through the salsa-smudged veil of history to answer such questions as: What do Cornish miners and their lunchtime pasties have to do with the taco? Did Glen Bell, of Taco Bell fame, actually invent the hard-shell version? Is a burrito also a taco? And how can a seemingly simple snack harbor so many mysteries? Grab a napkin, because things are going to get messy—and delicious—as we dive deep into the taco-verse to find out!

Learn more about your ad choices. Visit podcastchoices.com/adchoices

Eating to Win: Gatorade, Muscle Milk, and… Chicken Nuggets?

Cynthia Graber and Nicola Twilley · Gastropod

Ancient Greek Olympians swore by beans to give them a competitive edge. Japanese sumo wrestlers rely on a protein-rich soup called chankonabe to get into peak condition. And NBA all-stars Kevin Garnett, Carmelo Anthony, and Steph Curry credit their success to a pre-game PB&J. Throughout history, athletes have traditionally eaten something special they hope will give them an edge. But is there any science behind these special drinks and diets—and will consuming them help those of us who are not destined for sporting glory, too? Listen in this episode as we reveals the backstory behind such stadium staples as Gatorade and Muscle Milk—and the evidence for their efficacy. Learn more about your ad choices. Visit podcastchoices.com/adchoices

IN BRIEF

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): The study of groups is often one's first foray into advanced mathematics. In the naivete of set theory one develops tools for describing basic objects, and through a first run at analysis one develops a certain dexterity for manipulating symbols and definitions. But it is not until the study of groups that one must step back and inspect the larger picture. The main point of that picture (and indeed the main point of a group) is that algebraic structure can be found in the most unalgebraic of settings.

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): We present a video on Möbius transformations and the geometry of the sphere. Anyone who has taken or will take complex analysis (that means you engineers!) should watch this. It shows not only the beautiful correspondence between the two, but it reveals the intuition behind a lot of complex analysis, when more often than not a student is left in the dust of rigorous formulas. In short, this is a proof without words that the Möbius transformations are in correspondence with rigid motions of the unit sphere in $ \mathbb{R}^3$.

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): In this post we will assume the reader has a passing familiarity with some of the basic concepts of functional programming (the map, fold, and filter functions). We introduce these topics in our Racket primer, but the average reader will understand the majority of this primer without expertise in functional programming. Follow-ups to this post can be found in the Computational Category Theory section of the Main Content page. Preface: ML for Category Theory A few of my readers have been asking for more posts about functional languages and algorithms written in functional languages.

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): Last time we covered an operation in the LWE encryption scheme called modulus switching, which allows one to switch from one modulus to another, at the cost of introducing a small amount of extra noise, roughly $\sqrt{n}$, where n is the dimension of the LWE ciphertext. This time we'll cover a more sophisticated operation called key switching, which allows one to switch an LWE ciphertext from being encrypted under one secret key to another, without ever knowing either secret key.

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): My four-year-old son has declared 36 to be the best number. His reason: 36 is the only number he knows of that is both a square and a staircase number. And

GPUs on Fly.io are available to everyone!

Fly.io Blog

Fly.io makes it easy to spin up compute around the world, now including powerful GPUs. Unlock the power of large language models, text transcription, and image generation with our datacenter-grade muscle!

GPUs are now available to everyone!

We know you've been excited about wanting to use GPUs on Fly.io and we're happy to announce that they're available for everyone. If you want, you can spin up GPU instances with any of the following cards:

Ampere A100 (40GB) a100-40gb

Ampere A100 (80GB) a100-80gb

Lovelace L40s (48GB) l40s

To use a GPU instance today, change the vm.size for one of your apps or processes to any of the above GPU kinds. Here's how you can spin up an Ollama server in seconds:

Wrap text

Copy to clipboard

app = "your-app-name" region = "ord" vm.size = "l40s"

[http_service] internal_port = 11434 force_https = false auto_stop_machines = true auto_start_machines = true min_machines_running = 0 processes = ["app"]

[build] image = "ollama/ollama"

[mounts] source = "models" destination = "/root/.ollama" initial_size = "100gb"

Deploy this and bam, large language model inferencing from anywhere. If you want a private setup, see the article Scaling Large Language Models to zero with Ollama for more information. You never know when you have a sandwich emergency and don't know what you can make with what you have on hand.

We are working on getting some lower-cost A10 GPUs in the next few weeks. We'll update you when they're ready.

If you want to explore the possibilities of GPUs on Fly.io, here's a few articles that may give you ideas:

Deploy Your Own (Not) MidJourney Bot On Fly GPUs

Scaling Large Language Models to zero with Ollama

Transcribing on Fly GPU Machines

Depending on factors such as your organization's age and payment history, you may need to go through additional verification steps.

If you've been experimenting with Fly.io GPUs and have made something cool, let us know on the Community Forums or by mentioning us on Mastodon ! We'll boost the cool ones.

The Brightest Bulb

Cynthia Graber and Nicola Twilley · Gastropod

Imagine, for a moment, a world without garlic: garlic-free garlic bread, tzatziki sans Allium sativum, a chili crisp defanged. If this sounds like the makings of a horror story to you, you're not alone. Garlic consumption in the U. S. has quadrupled since 1980, and people around the world have been enjoying the stuff for thousands of years. But alliums smell like sulfur, and sulfur is something humans are born *not* liking—so why did we start adding garlic, onions, and their kin to our food? This episode, we join microbiologist Rob Dunn and food safety specialist Ben Chapman to follow along as they conduct the world's first experiment designed to figure out whether alliums started out as a food safety additive designed to keep our lamb stew safe for longer, and only later turned into a flavor we crave. Plus, why did the British government send garlic to the trenches in WWI? What do fetal sniffing, Egyptian fertility tests, Korean mythology, and the world's first-recorded labor strike have to do with the stinking rose? Listen in now for all this and more!

Learn more about your ad choices. Visit podcastchoices.com/adchoices

The Most Dangerous Fruit in America

Cynthia Graber and Nicola Twilley · Gastropod

Meet Saffron, the World's Most Expensive Spice

Cynthia Graber and Nicola Twilley · Gastropod

It's the poshest spice of all, often worth its weight in gold. But saffron also has a hidden history as a dye, a luxury self-tanner, and even a serotonin stimulant. That's right, this episode we're all about those fragile red threads plucked from the center of a purple crocus flower. Listen in as we visit a secret saffron field to discover why it's so expensive, talk to a clinical psychologist to explore the science behind saffron's reputation as the medieval Prozac, and explore the spice's off-menu role as an all-purpose beautifier for elites from Alexander the Great to Henry VIII. Learn more about your ad choices. Visit podcastchoices.com/adchoices

Dynamic Programming Fail

Jeremy Kun (Math ∩ Programming)

This is a story about a failure to apply dynamic programming to a woodworking project. I've been building a shed in my backyard, and for one section I decided to build the floor by laying 2x4 planks side by side. I didn't feel the need to join them with tongue-and-groove, but I did notice that using 2x4s alone wouldn't fit the width they were supposed to fill. I also had some 2x6 boards left over from a different part of the shed, and I realized that gave a neat dynamic programming problem: Can you fill a given width by laying planks of standard dimensional lumber?

It's the epitome of summertime: there's nothing like a cold, juicy slice of red watermelon on a swelteringly hot day. But, once upon a time, watermelons were neither red nor sweet—the wild watermelon has white flesh and a bitter taste. This episode, we scour Egyptian tombs, decaying DNA, and ancient literature in search of watermelon's origins. The quest for tasty watermelon continues into modern times, with the rediscovery of a lost (and legendarily sweet) varietal in South Carolina—and the Nigerian musical secret that might help you pick a ripe one. But the fruit's history has often been the opposite of sweet: watermelons have featured in some of the most ubiquitous anti-Black imagery in U. S. history. So how did the watermelon become the most dangerous—and racist—fruit in America?

Learn more about your ad choices. Visit podcastchoices.com/adchoices

The Scoop on Ice Cream

Cynthia Graber and Nicola Twilley · Gastropod

It's one of the most complex food products you'll ever consume: a thermodynamic miracle that contains all three states of matter—solid, liquid, and gas—at the same time. And yet no birthday party, beach trip, or Fourth of July celebration is complete without a scoop or two.

That's right—in this episode of Gastropod, we serve up a big bowl of delicious ice cream, topped with the hot fudge sauce of history and a sprinkling of science. Grab your spoons and join us as we bust ice-cream origin myths, dig into the science behind brain freeze, and track down a chunk of pricey whale poo in order to recreate the earliest published ice cream recipe. Learn more about your ad choices. Visit podcastchoices.com/adchoices

Where's the Beef? Lab-Grown Meat is Finally on the Menu

Cynthia Graber and Nicola Twilley · Gastropod

Can we really have our burger, eat it—and never need to kill a cow? Growing meat outside of animals—in a lab or, these days, in shiny steel bioreactors—promises to deliver a future in which we can enjoy sausages and sushi without guilt, and maybe even without sending our planet up in smoke. For years, it's seemed like science fiction, but it's finally a reality: this month, Americans will get their first chance to buy cultivated meat in a restaurant. But how exactly do you get chicken nuggets, BLTs, and bluefin sashimi from a bunch of cells growing in large metal vats? Does this new cultivated meat taste any good? Can enough be grown to replace industrial meat? And, if so, is this new technology actually an improvement on industrial animal agriculture and fishing? Gastropod is on the case! Join us this episode as we sink our teeth into a whole lot of lab-grown lunches, uncover the science behind the sci-fi, and investigate whether the companies making cultivated meat can actually fulfill the lofty promises they make.

Learn more about your ad choices. Visit podcastchoices.com/adchoices

The Pacific Wire

Vol. 1, No. 073 · 2026-03-30

Curated and composed automatically.